

Projeto AI

Algoritmos de Busca & *Reinforcement Learning*

Inteligência Artificial para Sistemas Autónomos

Profa. Anna Couto



Grupo 4:

Dinis Carraça Nº14058

José Feiteira Nº14300

Introdução

O projeto consiste na solução de um tabuleiro do jogo do 15, uma matriz 4x4 numerada de 1 a 15 com um quadrado vazio, este tem quatro ações possíveis para realizar o movimento: cima, baixo, esquerda e direita.

Dada a posição do quadrado alguns dos movimentos não podem ser realizados:

- Primeira fila: Não pode mover para cima;
- Primeira coluna: Não pode mover para a esquerda;
- Quarta fila: Não pode mover para baixo;
- Quarta coluna: Não pode mover para a direita

As possibilidades quando o quadrado está num dos cantos do tabuleiro são a junção das possibilidades acima descritas, assim:

- Superior Esquerdo: Não pode mover para cima e para a esquerda;
- Superior Direito: Não pode mover para cima e para a direita;
- Inferior Esquerdo: Não pode mover para baixo e para a esquerda;
- Inferior Direito: Não pode mover para baixo e para a direita;

Estas condicionantes serão úteis para encontrar as posições possíveis.

Cada tabuleiro tem duas soluções possíveis, uma com o quadrado vazio no fim, “1 2 3 ... 15 0”, e outra com este no início, “0 1 2 ... 14 15”.

A primeira parte deste projeto tem um foco em algoritmos de procura, não informados, *DFS*, *BFS* e *UCS*, e informados, *GBFS*, *A**.

Na segunda parte tentaremos solucionar os tabuleiros através de um modelo de *reinforcement learning*.

1. Algoritmos de Procura

1.1 Não Informados

Os algoritmos não informados realizam a procura de forma “cega”, isto é, apenas vão explorando o espaço com base no seu próprio método, sem olhar para outros fatores como distância ou heurísticas.

Para o nosso problema apenas testámos três destes algoritmos, o *BFS*, *DFS* e *UCS*, no entanto existem mais, i.e, o *IDS*, Iterative Deepening Search, seria outro algoritmo possível, este combina o *BFS* e o *DFS*, unindo o melhor dos dois mundos.

1.1.1 BFS

O *Breadth First Search*, *BFS*, funciona numa estrutura *FIFO*, “*First In First Out*”, como uma fila.

Neste o nó a ser expandido será o que foi primeiro gerado, assim leva a que o algoritmo aja como uma espécie de “onda”, expande todos os nós de cada nível antes de passar para o próximo.

Este algoritmo encontra quase sempre o caminho ótimo, no entanto é capaz de percorrer bastantes estados.

1.1.2 DFS

O segundo algoritmo implementado foi o *DFS*, ou *Depth First Search*, que funciona com uma estrutura de pilha, *LIFO*, “*Last In First Out*”.

Aqui o nó a ser expandido é sempre o que foi gerado em último lugar, fazendo com que o algoritmo percorra, muita das vezes, um caminho todo até ao fim e só depois é que retorna para o nó “mais à superfície”.

Quando comparado com o *BFS*, este tem a vantagem de poder ser mais rápido, contudo a solução apresentada pode não ser de todo a ótima.

1.1.3 UCS

O terceiro e último algoritmo foi o *Uniform Cost Search*, este escolhe o nó expandido com base no custo do caminho do nó atual até ao nó a ser expandido.

Para isto utiliza uma estrutura de fila prioritária, quem tem menor custo entra para a fila, tendo como objetivo encontrar o caminho com melhor custo-eficiência.

1.2 Informados

Ao contrário dos algoritmos anteriormente descritos, os informados escolhem o nó a ser expandido ao contabilizar uma heurística, isto é, uma estratégia que ajuda a guiar o algoritmo em prol de alcançar um objetivo definido.

Para o nosso projeto decidimos experimentar diferentes heurísticas até escolhermos a que ofereceu melhores resultados.

– 1ª Heurística: Número de peças fora do sítio

- Uma heurística simples que tal como o título sugere verificava apenas a quantidade de peças fora do sítio, sem contabilizar a distância
- Não seria uma métrica muito precisa visto que a peça fora do sítio tem sempre o mesmo valor, independentemente da distância a que estiver da posição desejada.

– 2ª Heurística: Distância de Manhattan

- Esta leva em conta o que tinha sido ignorado na anterior, a distância a que as peças estão da sua suposta posição.
- Por exemplo, após baralharmos o tabuleiro, a peça com o número encontra-se na posição (0,2), isto é, primeira fila e terceira coluna, como a posição objetivo desta seria (0,0) conseguimos chegar à conclusão de que a peça está a uma distância 3 de onde é suposto estar.

No fim decidimos utilizar a segunda heurística pelo equilíbrio entre uso e dificuldade.

Heurísticas como a distância Euclidiana e a Chebyshev não puderam ser contempladas visto que aceitam movimentos na diagonal, algo que não é possível na resolução do nosso puzzle. ADD FONTE

1.2.1 GBFS

O primeiro algoritmo implementado foi o *Greedy Base First Search*, este escolhe sempre o nó que tiver menor valor de heurística, neste caso o nó que está mais perto do objetivo.

Baseia-se em decisões locais, o que pode levar a que as soluções encontradas sejam nem sempre as melhores possíveis.

1.2.2 A*

O segundo e último algoritmo implementado foi o A*, este difere do GBFS visto que além de contabilizar a heurística para escolher o melhor caminho, também vai contabilizar o custo.

Este valor é obtido ao somar os dois valores, o algoritmo então compara este resultado e o nó que tiver o menor valor será o expandido.

O algoritmo será ótimo se:

- A função heurística for admissível, o *true cost* nunca é subestimado;
- A função heurística é consistente;
- O valor da heurística do nó atual é sempre menor ou igual à soma do valor desta no nó seguinte e o custo;

Teoricamente, este algoritmo será o que nos vai oferecer melhores resultados dentre todos os de procura. Mais à frente vamos ver se realmente foi o que aconteceu ou não.

2. Reinforcement Learning

A segunda abordagem utilizada para solucionar o nosso problema foi a do *Reinforcement Learning*.

Diferente dos algoritmos testados até agora, este método usufrui de uma certa autonomia na maneira como explora o espaço do tabuleiro, existe um agente que vai explorar o ambiente para chegar a um objetivo definido, parece bastante ao que foi antes descrito no entanto este agente é constituído pelos seguintes componentes:

1. *Modelo*: Permite prever o comportamento do ambiente para as ações possíveis, descritas no ponto 0. deste documento, o agente vai então utilizar estas previsões para decidir como agir;
2. *Policy*: Define regras que tem de tomar em certas situações, por exemplo se estiver no canto superior esquerdo do tabuleiro;
3. *Reward*: Ajuda a que o caminho escolhido seja sempre o melhor, acumulando menos reward cada vez que este não levou ao objetivo, quando for alcançado recebe um *reward* como “prenda” por ter tido sucesso.

Assim o modelo é primeiro treinado com valores do tabuleiro, quando este estiver concluído o agente então tentará encontrar uma solução.

O *reinforcement learning* tem como vantagens:

→ Aprender com cada utilização realizada, ou seja, se não encontrou a solução na primeira vez não quer dizer que não possa vir a encontrar à terceira, por exemplo;

→ Lida bem com situações onde os fatores são pouco previsíveis;

Apesar disto existem certos fatores que podem tornar este método numa escolha não tão certa:

→ Construir uma boa função de *reward* pode nem sempre ser fácil, o que vai comprometer a qualidade dos resultados;

→ O modo como explora o ambiente não é linear nem previsível, o agente pode tomar uma ação menos boa sem explicação aparente;

→ Dependendo do nosso ambiente o tempo de treino pode ser demasiado elevado;

Neste projeto utilizámos o *Q-Learning Model*, que encontra as ações com melhor qualidade, a partir do estado atual.

Através de tentativa-erro o modelo vai atualizando as melhores ações para o estado em questão, cada ciclo, tentativa de encontrar o caminho, tem o nome de episódio cada um termina quando chega ao objetivo, tabuleiro ordenado, ou quando o modelo fica preso, não existem mais ações possíveis mas não chegamos ao objetivo.

3. Comparação Entre Métodos

Vamos utilizar tabuleiros de 4 diferentes dificuldades, fácil, que resolvível entre 5 a 10 movimentos, médio, 20 a 40 movimentos, difícil, 60 a 70, e muito difícil, 200 movimentos no mínimo. Vamos testar cada método para o mesmo tabuleiro e avaliar a sua prestação.

Nas tabelas, i.e. tabela (1), a peça vazia é representada pelo número 0, e o objetivo ZF refere-se a “Zero no Final” e ZI a “Zero no Início”, tabela (2).

3.1 Tabuleiro Fácil

Do primeiro tabuleiro utilizado, tabela (1), conseguimos chegar aos seguintes resultados, tabela (2), para este teste foi considerado um limite máximo de dez mil tentativas para cada método.

Conseguimos notar que o único algoritmo de procura não informada que não conseguiu atingir o objetivo foi o *DFS*, o *BFS* e o *UCS* resolveram em 239 e 314 tentativas, respectivamente.

Ao contrário do que aconteceu anteriormente, todos os algoritmos de procura informada conseguiram alcançar o objetivo, tendo o melhor sido o *GBFS* com 7 tentativas.

Apesar disso, o método com melhor desempenho foi o Reinforcement Learning com 6.

Isto deve-se ao facto de que problemas com poucos movimentos, consequentemente menos complexidade, o método explora melhor este espaço reduzido.

3.2 Tabuleiro Médio

Para o tabuleiro presente na tabela (3), aumentamos o limite de tentativas para vinte mil, assim obtivemos os seguintes resultados.

Nos algoritmos não informados o *DFS* não conseguiu alcançar o objetivo, como aconteceu no ponto anterior, já o número de tentativas necessárias dos dois restantes métodos disparou para níveis na ordem das dezenas de milhares.

Quanto aos restantes métodos, a conclusão é a mesma da retirada no tabuleiro fácil, o Reinforcement Learning é o método mais eficiente.

3.3 Tabuleiro Difícil

No nível de dificuldade acima, chegamos ao tabuleiro visível na tabela (4), tendo aumentado para cinquenta mil o limite.

Como podemos notar na tabela (5), de entre todos os algoritmos de procura não informada, não houve um que conseguisse alcançar o objetivo, sendo possivelmente esta dificuldade, o ponto onde estes passam a ser obsoletos.

Os algoritmos informados continuam a conseguir chegar ao objetivo, com um número razoável de tentativas, na ordem dos milhares.

Surpreendentemente, o *Reinforcement Learning* não conseguiu encontrar a solução, perdendo assim o posto de método mais eficiente para o GBFS. Um dos possíveis motivos para isto seria o facto de que como a dificuldade do ambiente aumentou, o modelo do *RL* tem mais dificuldades em escolher o caminho certo.

3.4 Tabuleiro Muito Difícil

O último tabuleiro testado para os quatro métodos foi o presente na tabela (7), aumentamos o limite para quinhentas mil tentativas.

Como aconteceu no método anterior, as procuras não informadas e o *Reinforcement Learning* não conseguiram resolver o tabuleiro.

Apenas o GBFS e o A*, chegaram ao objetivo, 2379 e 265173 respectivamente, como podemos ver mesmo assim existe uma disparidade enorme entre os valores, mantendo o GBFS o estatuto de mais eficiente.

3.5 Tabuleiro Tie Braker

De modo a perceber em que dificuldade o A* começaria a não ser capaz de solucionar os tabuleiros decidimos testar os métodos num tabuleiro ainda mais difícil ao utilizado no ponto 3.4 deste documento, assim o tabuleiro utilizado está visível na tabela (9).

Como esperávamos o A* não conseguiu solucioná-lo, ao contrário o GBFS continua a conseguir solucionar tabuleiros com dificuldades elevadas, neste caso demorou 8449 tentativas.

Nota: A maioria das soluções foi encontrada com a solução do tabuleiro com o zero no final, a única exceção foi a encontrada pelo GBFS no ponto anterior, apenas encontrou a solução para o tabuleiro em questão com o zero no início. Em qualquer tentativa de solução os métodos tentaram sempre resolver com as duas hipóteses, ou seja, o resultado final compreende todas as opções possíveis.

4. Outros Métodos Possíveis

Para além dos métodos desenvolvidos nos pontos 1, 2 e 3, existem aqueles que não chegaram a ser desenvolvidos, alguns destes seriam capazes de solucionar o tabuleiro, outros nem por isso.

Assim os métodos não implementados, e a respetiva justificação de que seriam ou não viáveis, foram os seguintes:

1. *Redes Bayesianas*:
 - a. Funcionam com base nas probabilidades, e portanto, seria incapaz de tomar decisões visto que cada movimento no tabuleiro resulta num estado definitivo.
 - b. Não representam diretamente a sequência de ações de modo a atingir o objetivo, mas sim a relação estatística entre estados.
 - c. São maioritariamente utilizadas em espaços onde existe incerteza, o que não é o caso para o nosso problema.
2. *Markov Chains*:
 - a. Dependem, em questão de probabilidades, do estado atual para chegar ao estado seguinte, cada movimento não depende em termos probabilísticos do anterior para o nosso tabuleiro.
 - b. No caso de implementarmos o método mesmo assim, o tempo levado por este a tentar solucionar o tabuleiro seria extremamente elevado.
3. *Markov Decision Processes*
 - a. Adiciona ações, rewards e probabilidades na transição ao método anterior, o que o torna viável para decisões que estejam afetadas por um certo grau de incerteza, não é o nosso caso.
4. *Lógica de Primeira Ordem*
 - a. Exprime factos e relações, mas de uma forma declarativa, isto é, até poderia descrever as restrições presentes no nosso problema, mas nunca seria capaz de o resolver.
 - b. Codificar todos os movimentos possíveis iria criar um espaço de procura enorme.
5. *IDA**
 - a. Versão melhorada do A*, e o único dos métodos mencionados que realmente conseguiria solucionar o problema.
 - b. Lida melhor com espaços de procura com tamanho considerável.

5. Conclusões

Ao concluirmos todas as etapas do projeto, terminamos o mesmo com algumas conclusões.

Alguns algoritmos de procura, principalmente os não informados, apenas são úteis se o tabuleiro não tiver uma dificuldade muito elevada, para qualquer outra mostram-se completamente obsoletos.

Como podemos notar no gráfico (1), o método que se mostrou mais eficiente foi o *GBFS*, tendo sido o único a resolver todos os tabuleiros testados, segue-se do *A** que apenas cedeu no tabuleiro mais difícil de todos, isto mostra um claro domínio dos algoritmos de procura informada na nossa abordagem, o bom conjunto de regras definido, heurística definida em 1.2, pode ter sido a razão disso.

A abordagem de Reinforcement Learning demonstrou não lidar bem com o aumento da dificuldade, a expansão do espaço de procura, consequente deste facto, fez com que o método não conseguisse obter o melhor resultado na sua aprendizagem autónoma, fazendo com que nem sempre alcançasse o resultado desejado.

Assim podemos concluir que o método mais eficiente, e consequentemente o melhor para o nosso problema será o *GBFS*.

Contudo, vale salientar que todos os métodos tem uma utilidade válida.

Existem ainda outros métodos não implementados, mas dentre estes o único que seria capaz de se equiparar com os resultados obtidos seria o *IDA**, visto que nada mais é do que uma versão mais aprimorada do algoritmo que já implementámos, o *A**.

Apêndice

Nesta seção constam todos os gráficos, imagens e qualquer outro tipo de conteúdo visual, mencionados ao longo do documento.

1	2	3	4
5	6	7	8
9	10	0	15
13	14	12	11

Tabela (1) – Tabuleiro Fácil

Modelo	Num. Tentativas	Custo	Objetivo
BFS	239	X	zf
DFS	X	X	X
UCS	314	6	zf
GBFS	7	X	zf
A*	12	6	zf
RL	6	X	zf

Tabela (2) – Resultados M. Fácil

0	1	2	4
5	6	3	8
13	9	7	12
14	11	10	15

Tabela (3) – Tabuleiro Médio

Modelo	Num. Tentativas	Custo	Objetivo
BFS	14044	X	zf
DFS	X	X	X
UCS	13302	12	zf
GBFS	13	X	zf
A*	14	12	zf
RL	12	X	zf

Tabela (4) – Resultados M. Médio

5	1	3	4
2	11	7	8
13	6	0	10
14	15	12	9

Tabela (5) – Resultados M. Difícil

Modelo	Num. Tentativas	Custo	Objetivo
BFS	X	X	X
DFS	X	X	X
UCS	X	X	X
GBFS	1616	X	zf
A*	1909	24	zf
RL	X	X	X

Tabela (6) – Resultados M. Difícil

5	0	2	11
14	1	3	6
9	8	13	7
10	15	4	12

Tabela (7) – Tabuleiro Muito Difícil

Modelo	Num. Tentativas	Custo	Objetivo
BFS	X	X	X
DFS	X	X	X
UCS	X	X	X
GBFS	2379	X	zf
A*	265173	39	zf
RL	X	X	X

Tabela (8) – Resultados M. Difícil

14	13	15	7
11	12	9	5
6	0	2	1
4	8	10	3

Tabela (9) – Tabuleiro Impossível

Modelo	Num. Tentativas	Custo	Objetivo
BFS	X	X	X
DFS	X	X	X
UCS	X	X	X
GBFS	8499	X	zi
A*	X	X	X
RL	X	X	X

Tabela (10) – Resultados M. Difícil

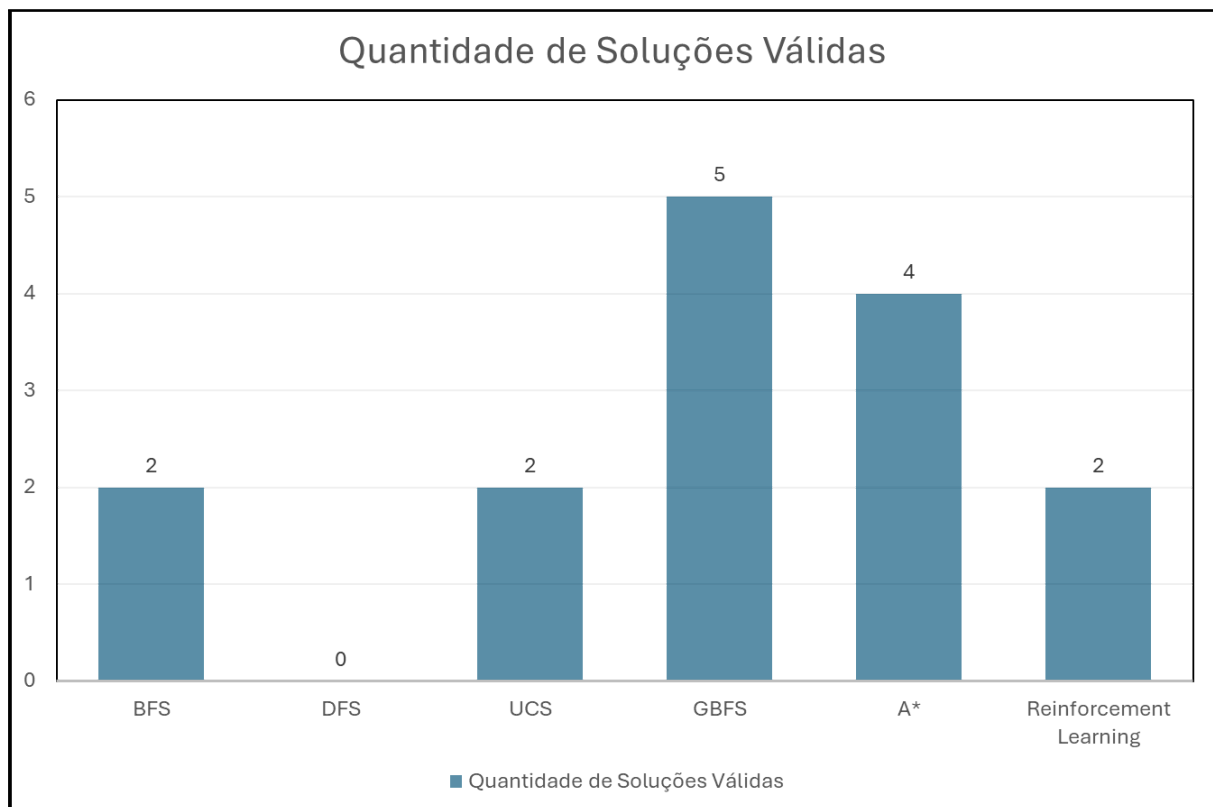


Gráfico (1) – Num. Sol. Válidas